

ParkFlow 360

Estructura de paquetes en Spring Boot con monolito hexagonal

Guia visual para explicar carpeta por carpeta, justificar la arquitectura y preparar la evolucion hacia MVC, DTO, JPA, puertos, adaptadores, errores y transacciones.

Spring Boot

REST + MVC

DTO + Validation

Hexagonal

Monolito modular

Objetivo docente: que el estudiante entienda que cada carpeta existe para proteger responsabilidades y evitar mezclar HTTP, negocio y persistencia.

La idea central: ordenar para no mezclar responsabilidades

POR QUE IMPORTA

Sin estructura

El controller termina validando negocio, instanciando objetos, manejando errores, devolviendo JSON y accediendo a datos. Funciona al inicio, pero se vuelve fragil.

Con estructura por capas y modulo

Cada paquete tiene una razon: dominio expresa negocio, aplicacion orquesta casos de uso e infraestructura conversa con el exterior.

Resultado esperado

El estudiante puede leer la ruta del codigo y explicar: que entra por HTTP, que valida el DTO, que decide el servicio y que objeto sale como respuesta.

Mensaje para clase: no estamos ordenando por estetica; estamos ordenando para que el sistema pueda crecer sin romperse.

La estructura actual que vamos a explicar

```
src/main/java/com/parkflow
├── ParkFlowApplication.java
├── shared
│   └── web
│       └── ApiErrorResponse.java
└── customer
    ├── domain
    │   └── Cliente.java
    ├── application
    │   └── Clienteservice.java
    └── infrastructure
        ├── adapter
        │   └── in
        │       └── web
        │           ├── ClienteController.java
        │           └── dto
        │               ├── CrearClienteRequest.java
        │               └── ClienteResponse.java
```

Lectura rapida del arbol

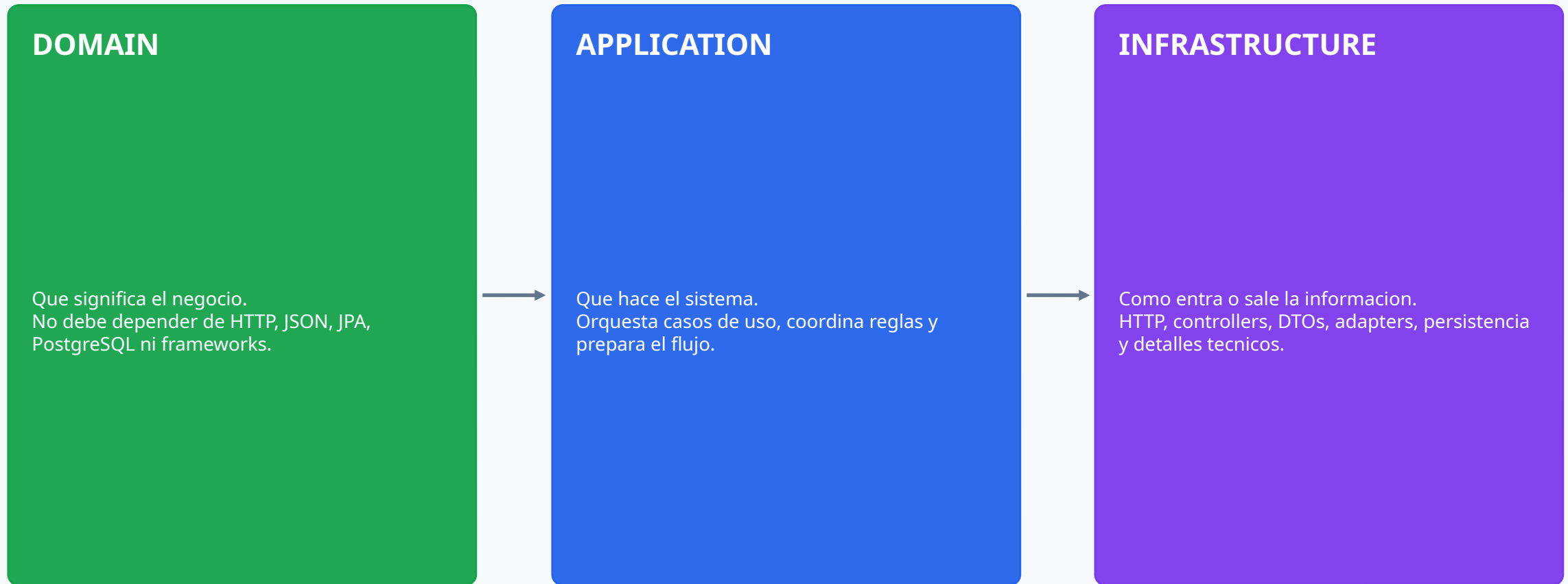
1. Existe un paquete raiz: com.parkflow.
2. shared contiene elementos reutilizables.
3. customer representa un modulo funcional.
4. Dentro de customer ya se separan dominio, aplicacion e infraestructura.
5. web y dto pertenecen a la entrada HTTP, no al negocio puro.

Interpretacion docente

Esta estructura es ideal para novatos porque permite empezar simple en memoria, pero sin enseñar una mala practica. Todavia no usamos puertos formales ni persistencia, pero ya estamos preparando el camino para arquitectura hexagonal.

Tres zonas mentales que deben dominar

MAPA BASE



Regla de oro: las capas externas pueden conocer hacia adentro; el nucleo no debe depender de los detalles externos.

src/main/java/com/parkflow: el paquete raiz

RAIZ

```
src/main/java/com/parkflow
├── ParkFlowApplication.java
├── shared
└── customer
```

Que representa

Es el punto comun de todo el backend. Desde aqui Spring Boot escanea componentes, controllers, services y configuraciones ubicadas debajo del paquete raiz.

Por que no poner todo junto

Porque un proyecto real no crece por archivos sueltos. Crece por modulos, responsabilidades y fronteras. El paquete raiz da orden, pero no debe convertirse en un basurero tecnico.

Como explicarlo al estudiante

com.parkflow es la ciudad. Dentro de la ciudad existen barrios: shared, customer, vehicle, billing, security, etc. Cada barrio tiene una responsabilidad clara.

ParkFlowApplication.java: punto de arranque

BOOTSTRAP

```
@SpringBootApplication
public class ParkFlowApplication {
    public static void main(String[] args) {
        SpringApplication.run(ParkFlowApplication.class, args);
    }
}
```

Que hace

Levanta el contexto de Spring, configura Tomcat embebido, activa auto-configuracion, registra beans y habilita el escaneo de componentes debajo del paquete raiz.

Que NO debe hacer

No debe contener logica de negocio, endpoints, reglas, validaciones de cliente ni acceso a datos. Es una puerta de entrada tecnica, no una clase de negocio.

Frase para clase

Si ParkFlowApplication empieza a crecer, estamos poniendo responsabilidad donde no corresponde.

shared/web: piezas reutilizables de la capa web

COMPARTIDO

```
shared
├── web
│   └── ApiResponse.java
```

Que significa shared

Contiene elementos que no pertenecen a un solo modulo funcional. Son transversales y pueden ser usados por customer, vehicle u otros modulos.

Por que web dentro de shared

Porque ApiResponse pertenece al contrato HTTP general. No es una regla de cliente ni de vehiculo: es una forma estandar de responder errores desde la API.

Justificacion arquitectonica

Centralizar respuestas de error evita que cada controller invente su propio formato. Mas adelante GlobalExceptionHandler usara este contrato para respuestas 400, 404, 409 y 500.

customer: modulo funcional, no simple carpeta

```
customer
├── domain
├── application
└── infrastructure
```

Que representa Customer

Es el modulo del negocio relacionado con clientes. Agrupa su modelo, su logica de aplicacion y sus formas de entrada/salida.

Por que modular por funcionalidad

Porque en sistemas reales el cambio ocurre por areas del negocio: cliente cambia por cliente, vehiculo cambia por vehiculo, pagos cambia por pagos.

Regla docente

No explicar customer como paquete tecnico. Explicarlo como una frontera funcional que protege cohesion: todo lo del cliente vive cerca, pero organizado por capas internas.

customer/domain: el negocio puro

DOMINIO

```
customer
└── domain
    └── Cliente.java
```

Que debe vivir aqui

Conceptos del negocio: Cliente, sus datos esenciales y reglas que no dependen de HTTP ni de base de datos.

Que NO debe vivir aqui

@RestController, @GetMapping, @RequestBody, ResponseEntity, @Entity, JpaRepository, SQL o clases de Postman. Todo eso es infraestructura.

Justificacion

Si el dominio se mantiene limpio, podremos probar reglas sin levantar Tomcat, sin Postman y sin PostgreSQL. Esa es una de las ganancias centrales del enfoque hexagonal.

Frase para repetir: el dominio no sabe como llega la petición ni donde se guarda el dato.

customer/application: orquesta el caso de uso

APLICACION

```
customer
└─ application
   └─ ClienteService.java
```

Funcion principal

Coordinar lo que el sistema hace: registrar, listar o buscar clientes. Recibe datos ya traducidos y ejecuta reglas de aplicacion.

En Capitulo 04

Puede trabajar con una lista en memoria para concentrar la clase en REST, DTOs, Validation y codigos HTTP, sin mezclar todavia PostgreSQL.

Evolucion posterior

En arquitectura hexagonal formal, este servicio implementara Ports IN y dependera de Ports OUT, no de JpaRepository directamente.

infrastructure/adapter/in/web: entrada HTTP

ADAPTER IN

```
custom er
└─ infrastructure
   └─ adapter
      └─ in
         └─ web
            └─ ClienteController.java
```

Por que esta tan profundo

Porque el controller es infraestructura. Es una forma concreta de entrar al sistema: HTTP + JSON + Spring MVC.

Responsabilidad del controller

Recibir la petición, leer PathVariable/RequestParam/RequestBody, activar @Valid, llamar al caso de uso y devolver un código HTTP coherente.

Lo que debe evitar

No debe contener SQL, no debe instanciar entidades JPA, no debe resolver reglas complejas, no debe repetir try/catch para cada error.

dto: contrato de entrada y salida de la API

DTO

```
web
├── ClienteController.java
└── dto
    ├── CrearClienteRequest.java
    └── ClienteResponse.java
```

CrearClienteRequest

Representa lo que el cliente HTTP envía. Aquí viven validaciones de formato: @NotBlank, @Size, @Email, @Pattern.

ClienteResponse

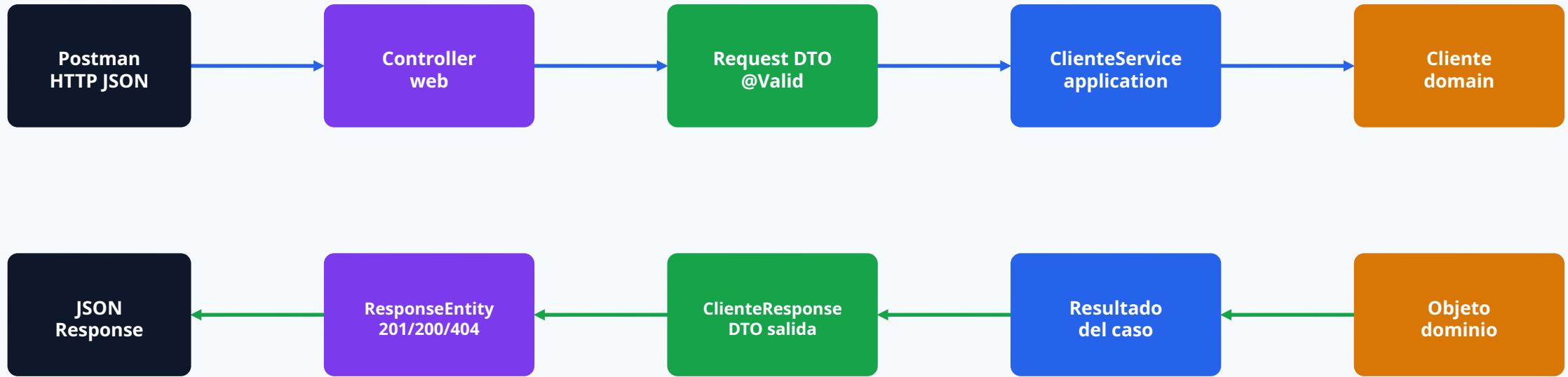
Representa lo que la API decide exponer. No necesariamente es igual al dominio y nunca debe ser una entidad JPA.

Justificación

Separar Request/Response protege el contrato externo. El dominio puede evolucionar sin romper automáticamente a los consumidores de la API.

Recorrido completo de una petición en esta estructura

FLUJO



La estructura de carpetas ayuda a ver el camino: web recibe, dto valida, application decide, domain representa negocio y web responde.

Por que esta estructura prepara los capitulos 04 al 08

EVOLUCION

Cap. 04	Spring MVC, DTO, Validation, ResponseEntity	Controller + dto + service en memoria
Cap. 05	PostgreSQL, JPA/Hibernate, Spring Data	Se agregara adapter/out/persistence
Cap. 06	Hexagonal formal: Port IN, Use Case, Port OUT	domain/port + application/service + adapters
Cap. 07	Relacion 1:N Cliente-Vehiculo	customer y vehicle colaboran por contratos
Cap. 08	Errores profesionales y transacciones	shared/web + exception handling + @Transactional

La carpeta actual no es definitiva: es una base didactica que permite evolucionar ordenadamente.

Errores que esta estructura ayuda a evitar

ANTI-PATRONES

Controller dios

Un controller que valida negocio, crea objetos, maneja errores y accede a datos viola separacion de responsabilidades.

Dominio contaminado

Poner @RestController, @Entity o ResponseEntity en el dominio mezcla negocio con infraestructura.

DTO confundido con entidad

Usar Cliente como Request o Response expone detalles internos y rompe el contrato cuando el modelo cambia.

shared como basurero

shared no es para cualquier cosa. Solo debe contener piezas realmente transversales y justificadas.

Persistencia prematura

Meter JPA/PostgreSQL antes de dominar REST hace que los estudiantes no sepan si el error es HTTP, validacion, Java o BD.

Paquetes por tecnologia

Separar solo por controllers/services/models sin modulo funcional dificulta crecer cuando aparecen customer, vehicle, payments y auth.

Guion para explicar carpeta por carpeta en clase

SCRIPT DOCENTE

1. Empezar por el arbol

Mostrar la imagen y preguntar: ¿que carpetas parecen de negocio y cuales parecen tecnicas?

2. Separar capas

Repetir: domain = negocio, application = caso de uso, infrastructure = detalles externos.

3. Seguir el flujo

Dibujar POST /api/clientes: Postman -> Controller -> DTO -> Service -> Cliente -> Response.

4. Justificar DTO

Explicar por que Request y Response no son Cliente ni ClienteJpaEntity.

5. Preparar evolucion

Anticipar que en el Cap. 05 aparece persistence, en el Cap. 06 ports y en el Cap. 08 shared/web gana mas valor.

6. Cerrar con defensa

Pedir que un estudiante explique que no debe vivir en domain y por que el controller es un adapter de entrada.

Cierre recomendado: "Una buena estructura no hace el sistema por nosotros, pero evita que lo hagamos mal desde el primer dia".

Preguntas de verificación para los estudiantes

DEFENSA ORAL

1 ¿Por que Cliente.java vive en domain y no en web/dto?

5 ¿Por que ClienteService no deberia depender de detalles HTTP?

2 ¿Que responsabilidad tiene ClienteController y que no debe hacer?

6 ¿Como evolucionaria esta estructura cuando agreguemos JPA y PostgreSQL?

3 ¿Por que ApiErrorResponse esta en shared/web?

7 ¿Por que esta organizacion ayuda a enseñar arquitectura hexagonal?

4 ¿Que diferencia hay entre CrearClienteRequest y ClienteResponse?

8 ¿Que error ocurre si mezclamos dominio con infraestructura?

Estas preguntas obligan a explicar estructura, responsabilidad y justificacion, no solo memorizar nombres de carpetas.

Mensaje final para tus estudiantes

La carpeta correcta no es decoracion: es una decision de arquitectura.

Cuando sabemos donde vive cada responsabilidad, podemos construir REST, validaciones, casos de uso, persistencia, relaciones y errores profesionales sin convertir el proyecto en codigo mezclado.

Dominio limpio

Aplicacion clara

Infraestructura controlada

Evolucion profesional